



From deployment to optimization, get practical Microsoft 365 and Copilot Studio insights at ESPC Microsoft 365 and AI Conference. Register today!

Analytics on Azure Blog

🔍 Search this community

Follow

Azure Databricks Cost Optimization

A Practical Guide to Reducing Costs
and Maximizing Performance



ANALYTICS ON AZURE BLOG 11 MIN READ

The Complete Guide to Azure Databricks Cost Optimization



Rafia Aqil  MICROSOFT

Aug 25, 2026

Co-Authored by: Aladdin Alchalabi [AladdinAlchalabi](#), Sanjeev Nair [Sanjeev Nair](#) and Rafia Aqil [Rafia Aqil](#)

This guide walks through a proven approach to Azure Databricks cost optimization, structured in three phases: **1.** Discovery, **2.** Cluster/Data/Code Best Practices, and **3.** Team Alignment & Next Steps.

Phase 1: Discovery

Assessing Your Current State

The following questions are designed to guide your initial assessment and help you identify areas for improvement. Documenting answers to each will provide a baseline for optimization and inform the next phases of your cost management strategy.



Find answers
Ask the agent

Environment & Organization	Cluster Management	Cost Optimization	Data Management	Performance Monitoring	Future Planning
What is the current scale of your Databricks environment? How many workspaces do you have? How are your workspaces organized (e.g., by environment type, region, use case)? How many clusters are deployed? How many users are active? What are the primary use cases for Databricks in your organization? • Data engineering • Data science • Machine learning • Business intelligence	How are clusters currently managed? • Manual configuration • Automated scripts • Databricks REST API • Cluster policies What is the average cluster uptime? • Hours per day • Days per week What is the average cluster utilization rate? • CPU usage • Memory usage	What is the current monthly spend on Databricks? • Total cost • Breakdown by workspace • Breakdown by cluster What cost management tools are currently in use? • Azure Cost Management • Third-party tools Are there any existing cost optimization strategies in place? • Reserved instances • Spot instances • Cluster auto-scaling	What is the current data storage strategy? • Data lake • Data warehouse • Hybrid What is the average data ingestion rate? • GB per day • Number of files What is the average data processing time? • ETL jobs • Machine learning models What types of data formats are used in your environment? • Delta Lake • Parquet • JSON • CSV • Other formats relevant to your workloads	What performance monitoring tools are currently in use? • Databricks Ganglia • Azure Monitor • Third-party tools What are the key performance metrics tracked? • Job execution time • Cluster performance • Data processing speed	Are there any plan expansions or changes to the Databricks environment? • New use cases • Increased data volume • Additional users What are the long-term goals for Databricks cost optimization? • Reducing overall spend • Improving resource utilization & cost attribution • Enhancing performance

Understanding Databricks Cost Structure

Total Cost = Cloud Cost + DBU Cost

- Cloud Cost: Compute (VMs, networking, IP addresses), storage (ADLS, MLflow artifacts) (firewalls), cluster type (serverless compute, classic compute)
- DBU Cost: Workload size, cluster/warehouse size, photon acceleration, compute runtime type (Jobs, Delta Live Tables, All Purpose Clusters, Serverless), model serving, queries per second, model



Find answers
Ask the agent

Diagnose Cost and Issues

Effectively diagnosing cost and performance issues in Databricks requires a structured approach. Use the following steps and metrics to gain visibility into your environment and uncover actionable insights.

1. Identify Costly Workloads

- [Account Console Usage Reports](#): Review usage reports to identify usage breakdowns by product, SKU name, and custom tags.
 - **Usage Breakdown by Product and SKU**: Helps you understand which services and compute types (clusters, SQL warehouses, serverless options) are consuming the most resources.
 - **Custom Tags for Attribution**: Tags allow you to attribute costs to teams, projects, or departments, making it easier to identify high-cost areas.
 - **Workflow and Job Analysis**: By correlating usage data with workflows and jobs, you can pinpoint long-running or resource-heavy workloads that drive costs.

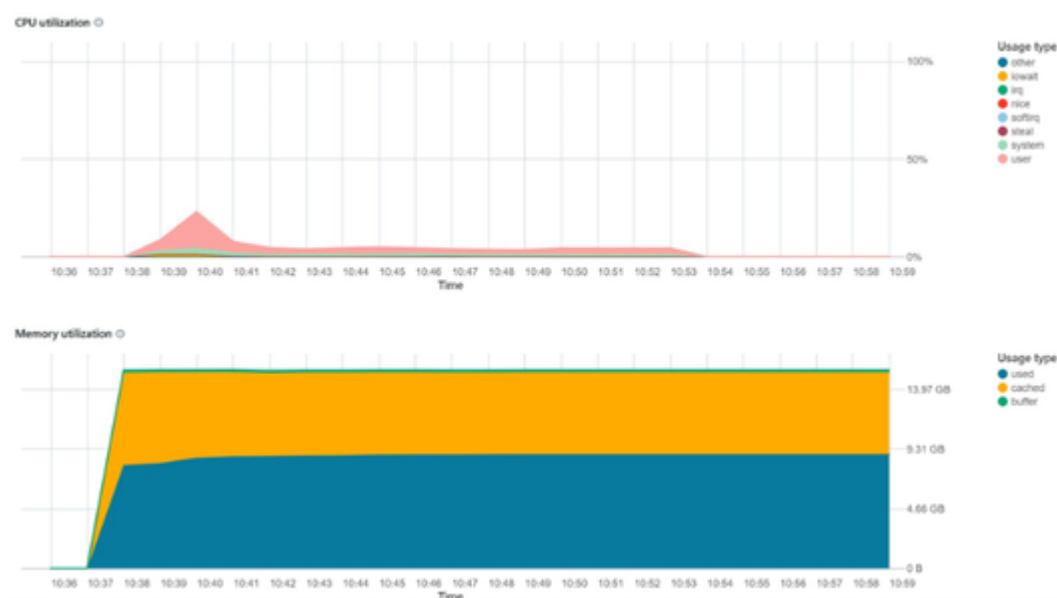
Focus on Long-Running Workloads: Examine workloads with extended runtimes or high resource utilization.

- **Key Question**: Which pipelines or workloads are driving the majority of your costs?
- [Governance Hub](#): is a centralized, account-level UI for monitoring and managing governance across Databricks. **Note** this is a beta feature, we will update this article as we get more information and use-cases on this feature.

Now That You've Identified Long-Running Workloads, Review These Key Areas:

2. Review Cluster Metrics

- **CPU Utilization**: Track guest, iowait, idle, irq, nice, softirq, steal, system, and user times to understand how compute resources are being used.
- **Memory Utilization**: Monitor used, free, buffer, and cached memory to identify over- or under-utilization.

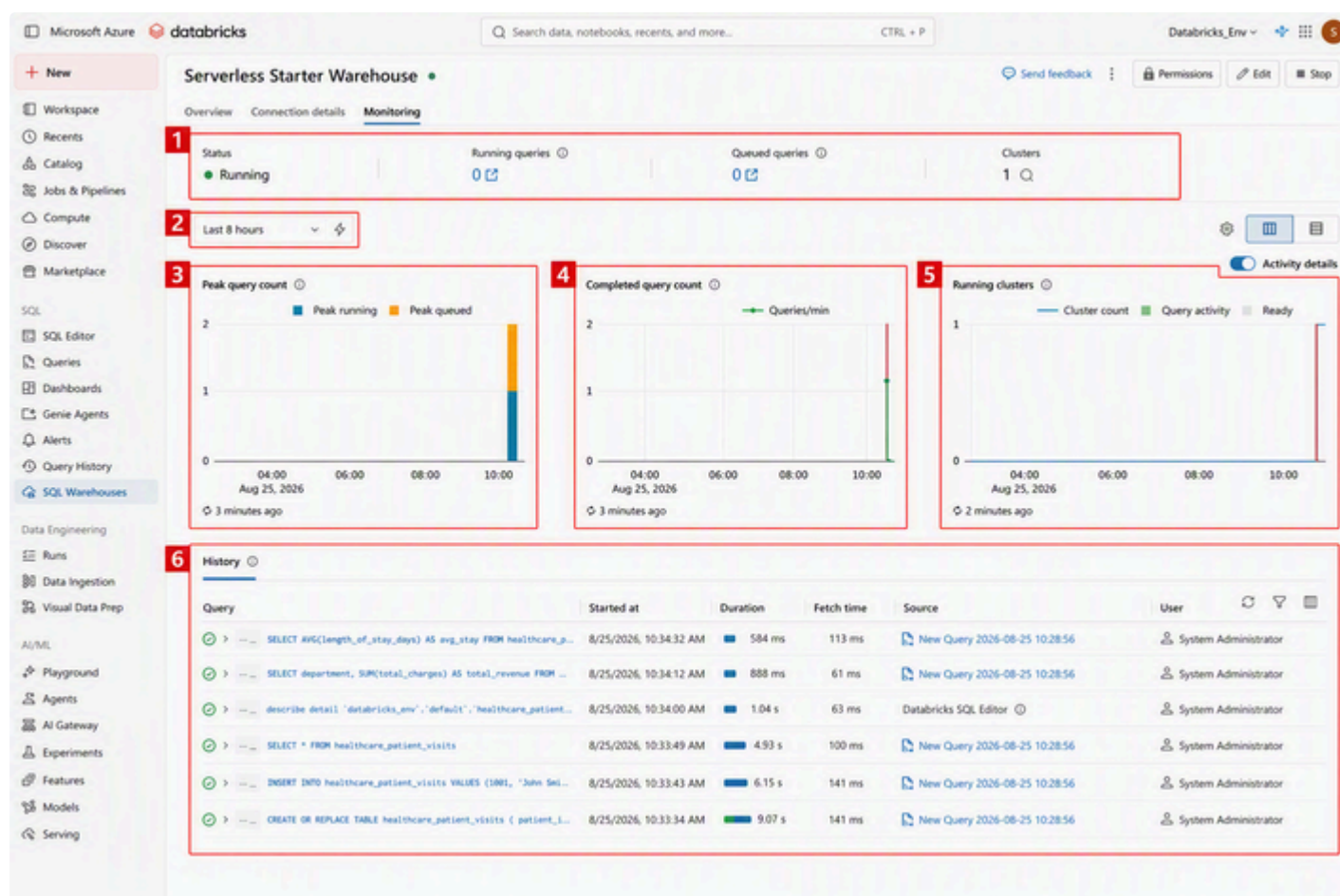


- **Key Question**: Is your cluster over- or under-utilized? Are resources being wasted or stretched too thin?

3. Review SQL Warehouse Metrics

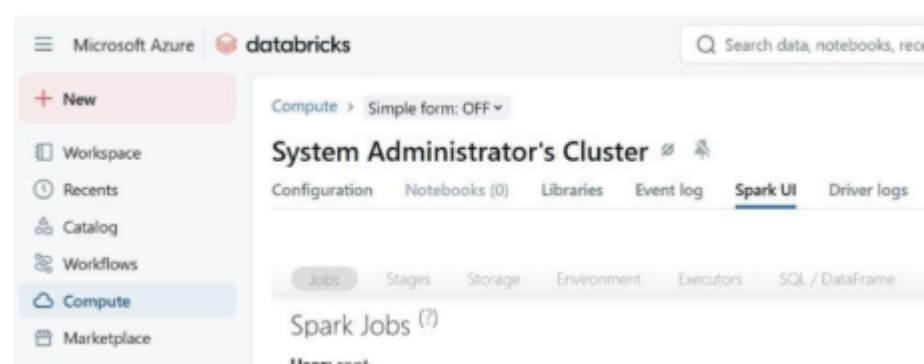


Find answers
Ask the agent



1. **Live Statistics:** Monitor warehouse status, running/queued queries, and current cluster count.
 2. **Time Scale Filter:** Analyze query and cluster activity over different time frames (8 hours, 24 hours, 7 days, 14 days).
 3. **Peak Query Count Chart:** Identify periods of high concurrency.
 4. **Completed Query Count Chart:** Track throughput and query success/failure rates.
 5. **Running Clusters Chart:** Observe cluster allocation and recycling events.
 6. **Query History Table:** Filter and analyze queries by user, duration, status, and statement type.
- **Key Question:** Is your SQL Warehouse over- or under-utilized? Are resources being wasted or stretched too thin?

4. Review Spark UI



- **Stages Tab:** Look for skewed data, high input/output, and shuffle times. Uneven task durations may indicate data skew or inefficient data handling.
- **Jobs Timeline:** Identify long-running jobs or stages that consume excessive resources.
- **Stage Analysis:** Determine if stages are I/O bound or suffering from data skew/spill.
- **Executor Metrics:** Monitor memory usage, CPU utilization, and disk I/O. Frequent garbage collection or high memory usage may signal the need for better resource allocation.

4.1. Spark UI: Storage & Jobs Tab

- **Storage Level:** Check if data is stored in memory, on disk, or both.
- **Size:** Assess the size of cached data.
- **Job Analysis:** Investigate jobs that dominate the timeline or have unusually long durations. Look for gaps caused by complex execution plans, non-Spark code, driver overload, or cluster malfunction.

4.2. Spark UI: Executor Tab



Find answers
Ask the agent

- **Storage Memory:** Compare used vs. available memory.
- **Task Time (Garbage Collection):** Review long tasks and garbage collection times.
- **Shuffle Read/Write:** Measure data transferred between stages.

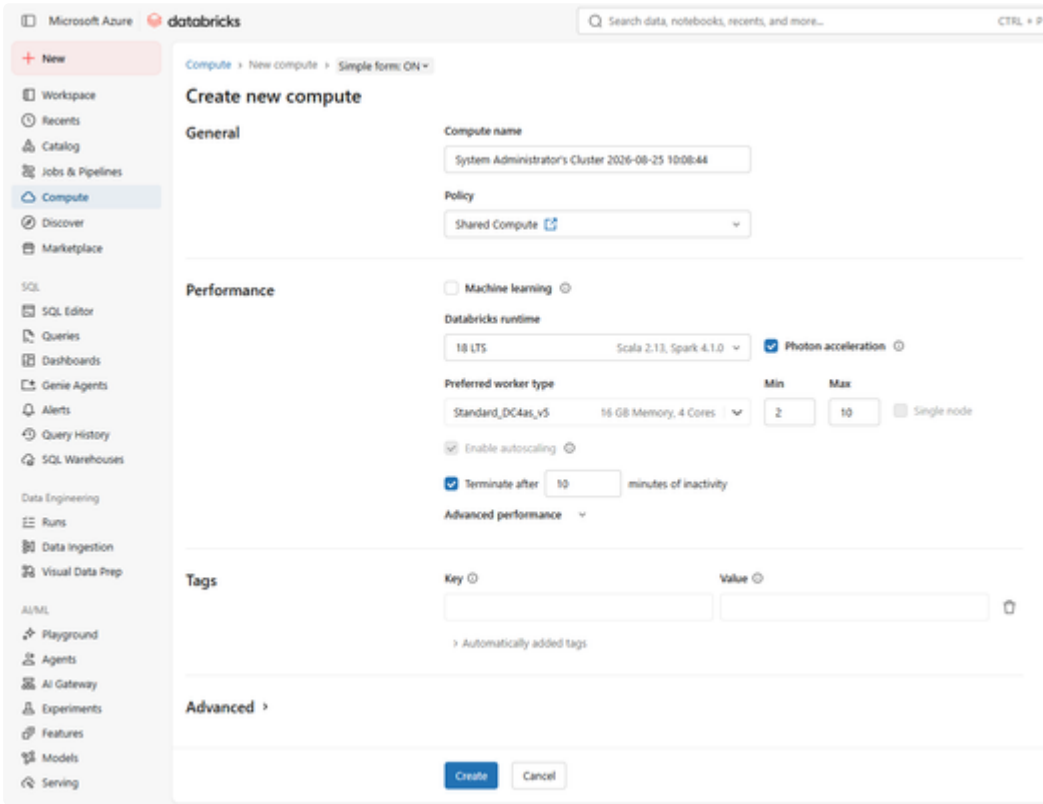
5. Additional Diagnostic Methods

- [System Tables in Unity Catalog](#) :
 - Query system tables for cost attribution and resource usage trends.
 - [Cost Observability Queries](#)
- **Tagging Analysis:**
Use tags to identify which teams or projects consume the most resources.
- **Dashboards & Alerts:**
Set up cost dashboards and budget alerts for proactive monitoring.

Phase 2: Cluster/Code/Data Best Practices Alignment

Cluster UI Configuration and Cost Attribution

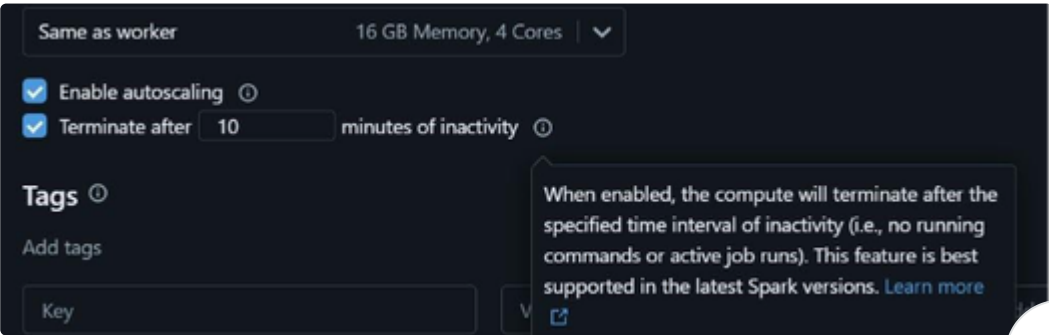
Effectively configuring clusters/workloads in Databricks is essential for balancing performance, scalability, and cost. Tuning settings and features when used strategically can help organizations maximize resource efficiency and minimize unnecessary spending.



Key Configuration Strategies

1. Reduce Idle Time: Clusters incur costs even when not actively processing workloads. To avoid paying for unused resources:

- **Enable Auto-Terminate:** Set clusters to automatically shut down after a period of inactivity. This simple setting can significantly reduce wasted spending.



2. Enable Autoscaling: Workloads fluctuate in size and complexity. Autoscaling allows clusters to dynamically adjust the number of nodes based on demand:



Find answers
Ask the agent

- **Automatic Resource Adjustment:** Scale up for heavy jobs and scale down for lighter loads, ensuring you only pay for what you use.

Performance

Databricks runtime version ⓘ
Runtime: 15.4 LTS (Scala 2.12, Spark 3.5.0) ▼

☐ Use Photon Acceleration ⓘ

Worker type ⓘ
Standard_DC4as_v5 16 GB Memory, 4 Cores ▼

Min workers: 1 Max workers: 8

Driver type
Standard_DS3_v2 14 GB Memory, 4 Cores ▼

☒ Enable autoscaling ⓘ

☒ Terminate after 15 minutes of inactivity ⓘ

- It significantly enhances cost efficiency and overall performance.
- For serverless and streaming, using Delta Live Tables with autoscaling is recommended.
- This approach leads to better resource management and reliability.

New SQL warehouse ✕

Name
SQL warehouse name

Cluster size ⓘ
X-Large 80 DBU / h / cluster ▼

Auto stop
☒ After 10 minutes of inactivity.

Scaling ⓘ
Min: 1 Max: 4 clusters (80 to 320 DBU)

Type
☒ Serverless ⓘ ☐ Pro ⓘ ☐ Classic

3. **Use Spot Instances:** For batch processing and non-critical workloads, spot instances offer substantial cost savings:

- **Lower VM Costs:** Spot instances are typically much cheaper than standard VMs. However, they are not recommended for jobs requiring constant uptime due to potential interruptions.

Min workers: 2 Max workers: 8

☒ Spot instances ⓘ

Use spot instances to save cost. If spot instances are evicted due to unavailability, on-demand instances will be deployed to replace evicted instances.

- **Considerations:**

- Azure Spot VMs are intended for non-critical, fault-tolerant tasks.
- They can be evicted without notice, risking production stability.
- No SLA guarantees mean potential downtime for critical applications.
- Using Spot VMs could lead to reliability issues in production environments.

4. **Leverage Photon Engine:** Photon is Databricks’ high-performance, vectorized query engine:

- **Accelerate Large Workloads:** Photon can dramatically reduce runtime for compute-intensive tasks, improving both speed and cost efficiency.

☒ Photon acceleration ⓘ

Min: 2 Max: 8

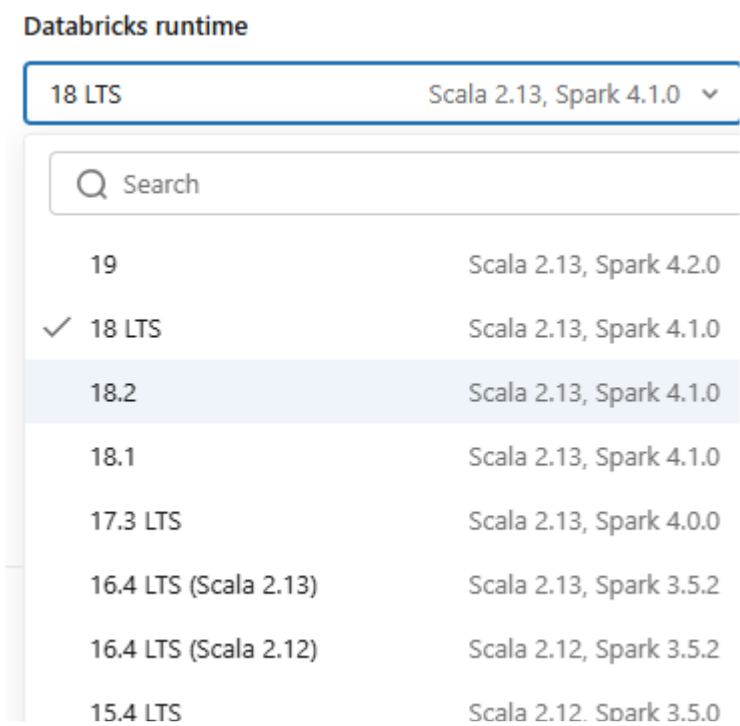
Photon accelerates modern Apache Spark workloads, reducing your total cost per workload.



Find answers
Ask the agent

5. **Keep Runtimes Up to Date:** Using the latest Databricks runtime ensures optimal performance and security:

- **Benefit from Improvements:** Regular updates include performance enhancements, bug fixes, and new features.

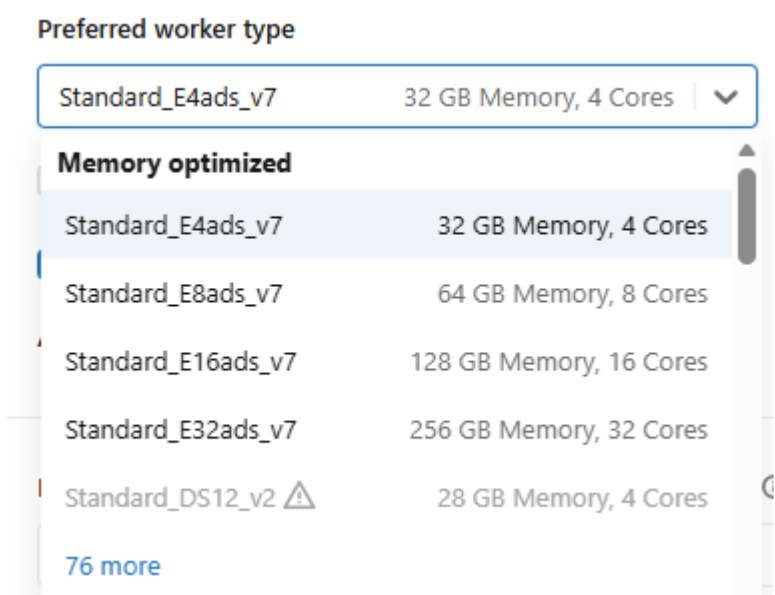


6. **Apply Cluster Policies:** Cluster policies help standardize configurations and enforce cost controls across teams:

- **Governance and Consistency:** Policies can restrict certain settings, enforce tagging, and ensure clusters are created with cost-effective defaults.

7. **Optimize Storage:** type impacts both performance and cost:

- **Switch from HDDs to SSDs:** SSDs provide faster caching and shuffle operations, which can improve job efficiency and reduce runtime.



8. **Tag Clusters for Cost Attribution:** Tagging clusters enables granular tracking and reporting:

- **Visibility and Accountability:** Use tags to attribute costs to specific teams, projects, or environments, supporting better budgeting and chargeback processes.

9. **Select the Right Cluster Type:** Different workloads require different cluster types, see table below for Serverless vs Classic Compute:

Feature	Classic Compute	Serverless Compute
Control	Full control over config & network	Minimal control, fully managed by Databricks
Startup Time	Slower (unless pre-warmed)	Instant



Find answers
Ask the agent

Cost Model	Hourly, supports reservations	Pay-per-use, elastic scaling
Security	VNet injection, private endpoints	NCC-based private connectivity
Best For	Heavy ETL, ML, compliance workloads	Interactive queries, unpredictable demand

- **Job Clusters:** Ideal for scheduled jobs and Delta Live Tables.
- **All-Purpose Clusters:** Suited for ad-hoc analysis and collaborative work.
- **Single-Node Clusters:** Efficient for simple exploratory data analysis or pure Python tasks.
- **Serverless Compute:** Scalable, managed workloads with automatic resource management.

11. Monitor and Adjust Regularly: review cluster metrics and query history:

- **Continuous Optimization:** Use built-in dashboards to monitor usage, identify bottlenecks, and adjust cluster size or configuration as needed.

Code Best Practices

1. Avoid Reprocessing Large Tables

- Use a CDC (Change Data Capture) architecture with Delta Live Tables (DLT) to process only new or changed data, minimizing unnecessary computation.

2. Ensure Code Parallelizes Well

- Write Spark code that leverages parallel processing. Avoid loops, deeply nested structures, and inefficient user-defined functions (UDFs) that can hinder scalability.

3. Reduce Memory Consumption

- Tweak Spark configurations to minimize memory overhead. Clean out legacy or unnecessary settings that may have carried over from previous Spark versions.

4. Prefer SQL Over Complex Python

- Use SQL (declarative language) for Spark jobs whenever possible. SQL queries are typically more efficient and easier to optimize than complex Python logic.

5. Modularize Notebooks

- Use %run to split large notebooks into smaller, reusable modules. This improves maintainability.

6. Use LIMIT in Exploratory Queries

- When exploring data, always use the LIMIT clause to avoid scanning large datasets unnecessarily.

7. Monitor Job Performance

- Regularly review Spark UI to detect inefficiencies such as high shuffle, input, or output. Review the below table for optimization opportunities:



Find answers
Ask the agent

High Shuffle Use Broadcast hash join to avoid data shuffling Try Shuffle hash join over sort-merge join Leverage cost-based optimizer to improve query plans	High Input •Use Delta format •Try Photon . It can help a lot with read speed, especially for wide tables. •Make your query more selective so it doesn't need to read as much data •Reconsider your data layout (Optimize , Z-Order , Partitioning , File Size Tuning so that data skipping is more effective ◦Data skipping: Dyamic file pruning (DFP) •If you're reading the same data multiple times, use the Delta cache . •If you're doing a join, consider trying to get DFP working
High Output •See if you have a merge that needs to be optimized •Use deletion vectors to mark existing rows as removed or changed without rewriting the Parquet file •Try Photon . Photon can help a lot with write speed	Long Duration & Low Shuffle/Input, Output • Reconsider your file format and Data layout (Optimize , Z-Order , Partitioning , File Size Tuning) • Comment out UDF and test performance ◦ Try commenting out your UDF to see how it impacts the speed of your pipeline. If the UDF is indeed where the time is being spent, your best bet is to rewrite the UDF using native functions.

For more guidance review: [Spark stage high I/O - Azure Databricks | Microsoft Learn](#)

Databricks Code Performance Enhancements & Data Engineering Best Practices

By enabling the below features and applying best practices, you can significantly lower costs, accelerate job execution, and build Databricks pipelines that are both scalable and highly reliable. For more guidance review: [Comprehensive Guide to Optimize Data Workloads | Databricks](#).

Feature / Technique	Purpose / Benefit	How to Use / Enable / Key Notes
Disk Caching	Accelerates repeated reads of Parquet files	Set spark.databricks.io.cache.enabled = true
Dynamic File Pruning (DFP)	Skips irrelevant data files during queries, improves query performance	Enabled by default in Databricks
Low Shuffle Merge	Reduces data rewriting during MERGE operations, less need to recalculate ZORDER	Use Databricks runtime with feature enabled
Adaptive Query Execution (AQE)	Dynamically optimizes query plans based on runtime statistics	Available in Spark 3.0+, enabled by default
Deletion Vectors	Efficient row removal/change without rewriting entire Parquet file	Enable in workspace settings, use with Delta Lake
Materialized Views	Faster BI queries, reduced compute for frequently accessed data	Create in Databricks SQL
Optimize	Compacts Delta Lake files, improves query performance	Run regularly, combine with ZORDER on high-I/O
ZORDER	Physically sorts/co-locates data by chosen columns for faster queries	Use with OPTIMIZE, set

**Find answers**
Ask the agent

Auto Optimize	Automatically compacts small files during writes	Enable optimizeWrite and autoCompact table
Liquid Clustering	Simplifies data layout, replaces partitioning/ZORDER, flexible clustering keys	Recommended for new Delta tables, enables e
File Size Tuning	Achieve optimal file size for performance and cost	Set delta.targetFileSize table property
Broadcast Hash Join	Optimizes joins by broadcasting smaller tables	Adjust spark.sql.autoBroadcastJoinThreshold a
Shuffle Hash Join	Faster join alternative to sort-merge join	Prefer over sort-merge join when broadcasting
Cost-Based Optimizer (CBO)	Improves query plans for complex joins	Enabled by default, collect column/table statist
Data Spilling & Skew	Handles uneven data distribution and excessive shuffle	Use AQE, set spark.sql.shuffle.partitions=auto,
Data Explosion Management	Controls partition sizes after transformations (e.g., explode, join)	Adjust spark.sql.files.maxPartitionBytes, use rep
Delta Merge	Efficient upserts and CDC (Change Data Capture)	Use MERGE operation in Delta Lake, combine v
Data Purging (Vacuum)	Removes stale data files, maintains storage efficiency	Run VACUUM regularly based on transaction fi

Phase 3: Team Alignment and Next Steps

Implementing Cost Observability and Taking Action

Effective cost management in Databricks goes beyond configuration and code—it requires robust observability, granular tracking, and proactive measures. Below outlines how your teams can achieve this using system tables, tagging, dashboards, and actionable scripts.

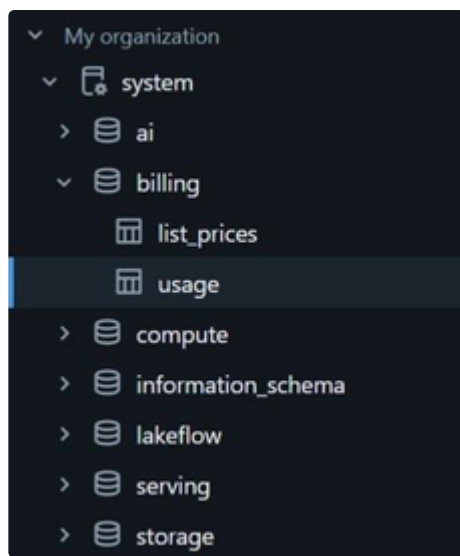
1. Cost Observability with System Tables

Databricks Unity Catalog provides system tables that store operational data for your account. These tables enable historical cost observability and empower FinOps teams to analyze spend independently.

- **System Tables Location:** Found inside the Unity Catalog under the “system” schema.
- **Key Benefits:** Structured data for querying, historical analysis, and cost attribution.
- **Action:** Assign permissions to FinOps teams so they can access and analyze dedicated cost tables.



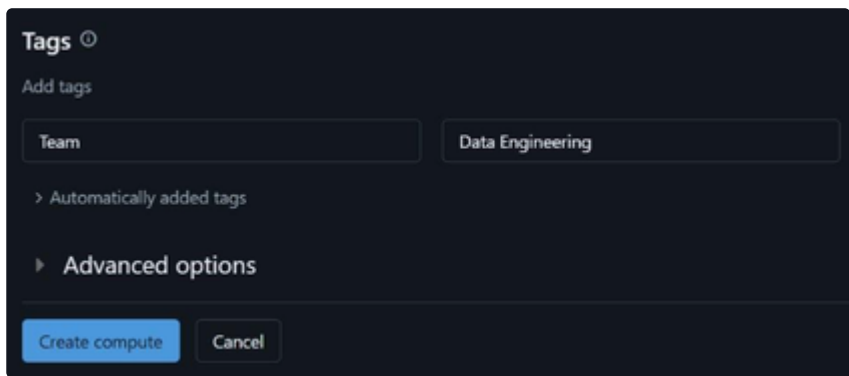
Find answers
Ask the agent



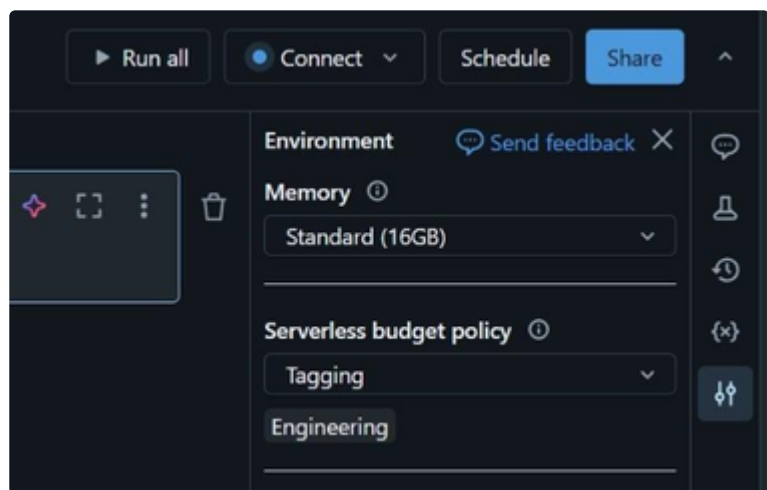
2. Enable Tags for Granular Tracking

Tagging is a powerful feature for tracking, reporting, and budgeting at a granular level.

- **Classic Compute:** Manually add key/value pairs when creating clusters, jobs, SQL Warehouses, or Model Serving endpoints. Use cluster policies to enforce custom tags.



- **Serverless Compute:** Create budget policies and assign permissions to teams or members for serverless workloads.



- **Action:** Tag all compute resources to enable detailed cost attribution and reporting.

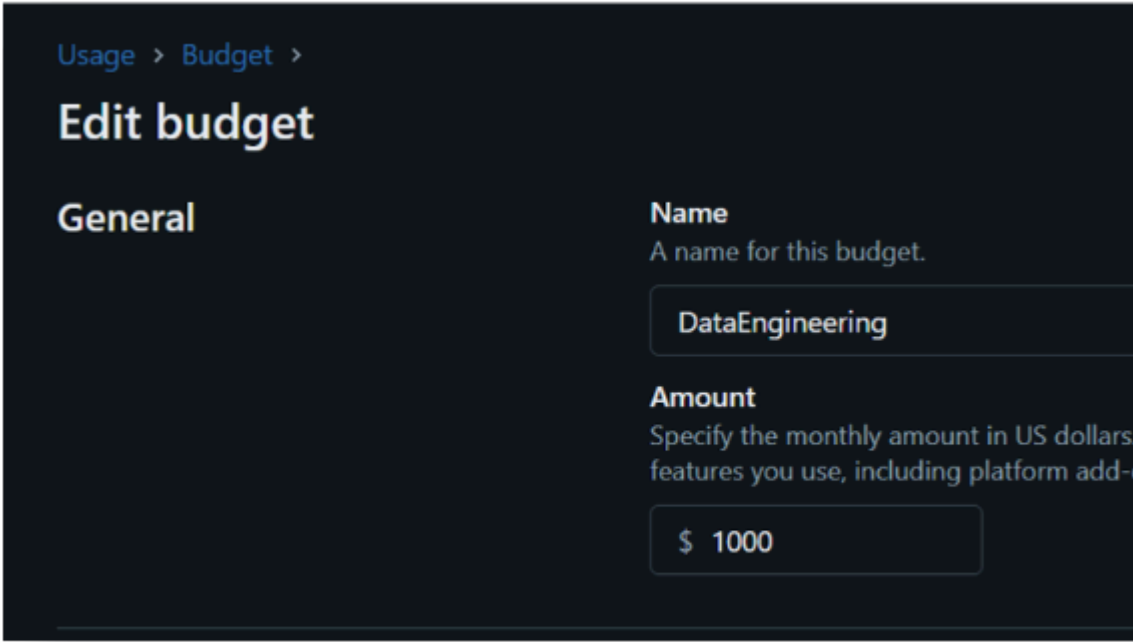
3. Track Costs with Dashboards and Alerts

Databricks offers prebuilt dashboards and queries for cost forecasting and usage analysis.

- [Dashboards](#): Visualize spend, usage trends, and forecast future costs.
- [Prebuilt Queries](#): Use top queries with system tables to answer meaningful cost questions.
- [Budget Alerts](#): Set up alerts in the Account Console (Usage > Budget) to receive notifications when spend approaches defined thresholds.



Find answers
Ask the agent



4. Build Culture of Efficiency

To go beyond technical fixes and build a culture of efficiency, by focusing on the below strategic actions:

- **Collaborate with Internal Engineers:**
Spend time with engineering teams to understand workload patterns and optimization opportunities.
- **Peer Reviews and Code Audits:**
Conduct regular code review sessions and peer reviews to ensure best practices are followed for Spark jobs, data pipelines, and cluster configurations.
- **Create Internal Best Practice Documentation:**
Develop clear guidelines for writing optimized code, managing data, and maintaining clusters. Make these resources easily accessible for all teams.
- **Implement Observability Dashboards:**
Use Databricks’ built-in features to create dashboards that track spend, monitor resource utilization, and highlight anomalies.
- **Set Alerts and Budgets:**
Configure alerts for long-running workloads and establish budgets using prebuilt Databricks capabilities to prevent cost overruns.

5. Azure Reservations and Azure Savings Plan

When optimizing Databricks costs on Azure, it’s important to understand the two main commitment-based savings options: Azure Reservations and Azure Savings Plans. Both can help you reduce compute costs, but they differ in flexibility and how savings are applied.

- **Which Should You Choose?**
 - **Reservations** are ideal if you have stable, predictable Databricks workloads and want maximum savings.
 - **Savings Plans** are better if you expect your compute needs to change, or if you want a simpler, more flexible way to save across multiple services.

Pro Tip: You can combine both options—use Reservations for your baseline, always-on Databricks clusters, and Savings Plans for bursty, variable, or new workloads.

Summary Table: Action Steps

It’s critical to monitor costs continuously and align your teams with established best practices, while scheduling regular code review sessions to ensure efficiency and consistency.

Area	Best Practice / Action
System Tables	Use for historical cost analysis and attribution

**Find answers**
Ask the agent

Tagging	Apply to all compute resources for granular tracking
Dashboards	Visualize spend, usage, and forecasts
Alerts	Set budget alerts for proactive cost management
Scripts/Queries	Build custom analysis tools for deep insights
Cluster/Data/Code Review & Align	Regularly review best practices, share findings, and align teams on optimization
Save on your Usage	Consider Azure Reservations and Azure Savings Plan

Updated Sep 21, 2026

VERSION 3.0

ANALYTICS

AZURE

AZURE DATABRICKS

+ TAG

 2

 Comment



Rafia Aqil  MICROSOFT

Joined June 13, 2025

[View Profile](#)



Analytics on Azure Blog

Follow this blog board to get notified when there's new activity

 **Follow**



Leave a comment...

Share this page

- 
- 
- 
- 
- 
- 
- 

What's new

- Surface Pro
- Surface Laptop
- Surface Laptop Studio 2
- Copilot for organizations



Find answers

Ask the agent

Copilot for personal use

AI in Windows

Explore Microsoft products

Windows 11 apps

Microsoft Store

Account profile

Download Center

Microsoft Store support

Returns

Order tracking

Certified Refurbished

Microsoft Store Promise

Flexible Payments

Education

Microsoft in education

Devices for education

Microsoft Teams for Education

Microsoft 365 Education

How to buy for your school

Educator training and development

Deals for students and parents

AI for education

Business

Microsoft AI

Microsoft Security

Dynamics 365

Microsoft 365

Microsoft Power Platform

Microsoft Teams

Microsoft 365 Copilot

Small Business

Developer & IT

Azure

Microsoft Developer

Microsoft Learn

Support for AI marketplace apps

Microsoft Tech Community

Microsoft Marketplace

Marketplace Rewards

Visual Studio

Company

Careers



Find answers

Ask the agent

[About Microsoft](#)

[Company news](#)

[Privacy at Microsoft](#)

[Investors](#)

[Diversity and inclusion](#)

[Accessibility](#)

[Sustainability](#)



[Your Privacy Choices](#)

[Consumer Health Privacy](#)

[Sitemap](#)

[Contact Microsoft](#)

[Privacy](#)

[Manage cookies](#)

[Terms of use](#)

[Trademarks](#)

[Safety & eco](#)

[Recycling](#)

[About our ads](#)

[© Microsoft](#)



Find answers

Ask the agent